

config > JS db.js > ...

```
1  import mongoose from "mongoose";
2
3
4  async function connectDB(){
5      try{
6          const connectdb = await mongoose.connect("mongodb://127.0.0.1:27017/StudentManagementSystem");
7
8          console.log("db connected");
9
10     }catch(error){
11         console.log(error.message);
12     }
13 }
14
15
16 export default connectDB;
```

middleware > **JS** export.js > default

```
1  class HttpError extends Error{
2    constructor(message,statusCode){
3      super(message),
4      this.statusCode = statusCode,
5      this.name = "HttpError"
6    }
7  }
8
9  export default HttpError;
```

JS server.js > ...

```
1  import express from "express";
2  import HttpError from "../middleware/export.js";
3  import studentRoutes from "../routes/studentRoutes.js";
4  import connectDB from "../config/db.js";
5
6  const app = express();
7
8  const port = 5000;
9
10 app.use(express.json());
11
12 // routes
13 app.use("/student", studentRoutes);
14
15 app.get("/", (req, res) => {
16   res.json({ message: "hello from server" });
17 });
18
19 // undefined routes
20 app.use((req, res, next) => {
21   next(new HttpError("requested route not found", 404));
22 });
23
24 // centralized error
25 app.use((error, req, res, next) => {
26   if (res.headersSent) {
27     return next(error);
28   }
29 });
```

```
29
30     res.status(error.statusCode || 500).json({
31       message: error.message || "internal server error",
32     });
33   });
34
35   async function startServer() {
36     try {
37       await connectDB();
38
39       app.listen(port, () => {
40         console.log(`server running on port ${port}`);
41       });
42     } catch (error) {
43       console.log(error.message);
44       process.exit(1);
45     }
46   }
47
48   startServer();
```

modal > JS student.js > studentSchema > mobileNumber > minlength

```
1  import mongoose from "mongoose";
2
3  const studentSchema = new mongoose.Schema({
4
5      name:{
6          type:String,
7          required:true,
8          trim:true
9      },
10     grId:{
11         type:Number,
12         required:true,
13         unique:true
14     },
15     email:{
16         type:String,
17         required:true,
18         unique:true
19     },
20     course:{
21         type:String,
22         enum:["fullStack","graphic design","UI/UX"],
23         required:true
24     },
25     isActive:{
26         type:String,
27         enum:["active","pending","hold","suspend"],
28         default:"active"
```

```
20  ✓ course:{
21      type:String,
22      enum:["fullStack","graphic design","UI/UX"],
23      required:true
24  },
25  ✓ isActive:{
26      type:String,
27      enum:["active","pending","hold","suspend"],
28      default:"active"
29  },
30  ✓ mobileNumber:{
31      type:Number,
32      minlength:10,
33      required:true
34  }
35  });
36
37
38  const student = mongoose.model("StudentData",studentSchema);
39
40  export default student;
```

controller > JS StudentController.js > [🔍] updateDataManually > [🔍] isValidUpdate > [🔍] updates.every() callback

```
1  import { HttpError } from "../middleware/export.js";
2  import { student } from "../model/student.js";
3
4  // Add Student
5  const add = async (req, res, next) => {
6    try {
7      const { name, grId, email, course, isActive, mobileNumber } = req.body;
8
9      const newStudent = new student({
10        name,
11        grId,
12        email,
13        course,
14        isActive,
15        mobileNumber,
16      });
17
18      await newStudent.save();
19
20      res.status(201).json({
21        success: true,
22        message: "Student added successfully",
23        newStudent,
24      });
25    } catch (error) {
26      next(new HttpError(error.message, 500));
27    }
28  };
29
```

controller > JS StudentController.js > updateDataManually > isValidUpdate > updates.every() callback

```
30 // Get All Students
31 const getAllStudentData = async (req, res, next) => {
32   try {
33     const students = await student.find({});
34
35     if (students.length <= 0) {
36       return res.status(200).json({
37         success: true,
38         message: "No student found",
39       });
40     }
41
42     res.status(200).json({
43       success: true,
44       total: students.length,
45       students,
46     });
47   } catch (error) {
48     next(new HttpError(error.message, 500));
49   }
50 };
51
52 // Get Student By ID
53 const getStudentById = async (req, res, next) => {
54   try {
55     const { id } = req.params;
56
57     const studentData = await student.findById(id);
```



```
56 const getStudentsById = async (req, res, next) => {
57   const studentData = await student.findById(id);
58
59   if (!studentData) {
60     return next(new HttpError("Student not found", 404));
61   }
62
63   res.status(200).json({
64     success: true,
65     studentData,
66   });
67 } catch (error) {
68   next(new HttpError(error.message, 500));
69 }
70 };
71
72 // Delete Student
73 const deleteStudent = async (req, res, next) => {
74   try {
75     const { id } = req.params;
76
77     const deletedStudent = await student.findByIdAndDelete(id);
78
79     if (!deletedStudent) {
80       return next(new HttpError("Student not found", 404));
81     }
82
83     res.status(200).json({
84       success: true
```